

ARGUS

Autonomous Regional Geospatial Urban Sentinel

A Multimodal Geospatial Intelligence Platform for Closed-Loop Urban Infrastructure & Public-Safety Management

Pilot Zone: VIT Bhopal University Campus + Kothri, Madhya Pradesh

Architecture: Zero recurring cost · Local-first inference · Fully open-source

Core Stack: PyTorch · PostGIS · FastAPI · CesiumJS · PyTorch Geometric (GNN)

MONITORS FIVE REAL URBAN HAZARD CLASSES



Potholes & Road
Damage



Garbage & Illegal
Dumping



Unsafe Construction
Sites



Fallen Trees / Debris



Exposed Electric
Lines

What This Document Covers

This is the complete, presentation-ready specification for ARGUS. It has been redesigned around five real, co-occurring hazard classes found in any Indian campus or municipal ward — not potholes alone — while keeping the research core (a graph neural network that predicts risk before it's reported) and the zero-cost engineering discipline that makes it buildable in one semester.

Why five hazard classes instead of one

A pothole, an overflowing garbage point, an unfenced construction pit, a fallen branch, and a sparking open wire are different physically — but operationally identical: someone must detect it, someone must judge how dangerous it is, someone must decide who fixes it first, and someone must confirm it actually got fixed. ARGUS is one intelligence layer underneath all five, not five separate apps.

Contents

01	Executive Overview	3
02	The Five Hazard Classes	4
03	Problem, Motivation & Research Positioning	5
04	Objectives & Success Metrics	6
05	System Architecture	7
06	AI / ML Methodology (per hazard class)	8
07	The Predictive Engine — Spatio-Temporal GNN	10
08	Fusion & the ARGUS Risk Score	11
09	Geospatial Intelligence Layer	12
10	Dataset & Annotation Plan	13
11	Security, Privacy & Ethics	14
12	Zero-Cost Technology Stack	15
13	Data Model & API Blueprint	16
14	Evaluation & Research Plan	17
15	20-Week Implementation Roadmap	18
16	Deliverables & Live Demo Script	19
17	Risk Register & Zero-Cost Proof	20
18	Final Vision & Recommended Title	21

Executive Overview

ARGUS is a closed-loop geospatial decision-intelligence platform for urban infrastructure and public-safety hazards. It fuses ground-level citizen photographic evidence, passive civic text signals, satellite/environmental context, weather data, road-network topology, and historical hazard trajectories into one explainable operational record per hazard — regardless of which of the five hazard classes it belongs to.

Every stage produces structured output that feeds the next: perception feeds severity, severity feeds a transparent risk score, risk feeds a spatio-temporal forecast, forecasts feed prioritised dispatch, and every closed case re-enters the historical dataset that makes the next forecast better. Nothing in the loop is a dead end.



Three Escalating Questions ARGUS Answers

Descriptive

"What is happening right now?"
— live multi-hazard map,
severity, confidence, evidence.

Predictive

"What is likely to happen next?" —
7-day and 30-day spatio-temporal
risk forecasts per road segment.

Prescriptive

"What should we do, with the crews
we actually have?" — priority-aware
dispatch and a what-if simulator.

Why this raises the ceiling on a final-year project

Most student infrastructure projects stop at descriptive detection of a single hazard type. ARGUS is built so its headline contribution — a Graph Neural Network that propagates risk across the road network, across all five hazard classes simultaneously — is genuinely defensible as research, while every other subsystem remains buildable by one engineer in one semester on entirely free tooling.

Pilot geography is intentionally constrained to VIT Bhopal and Kothri, keeping data collection, annotation, and live demonstration tractable while the architecture — schema, service boundaries, API contracts — is written to scale to a full municipal ward without redesign.

The Five Hazard Classes

ARGUS is deliberately not a "pothole app." The pilot area — like any real campus or ward — produces five recurring hazard patterns at once. Each gets its own detector head and its own risk logic, but shares one fusion layer, one map, one dispatch system, and one verification workflow.

Potholes & Road Surface Damage

Vision · Depth proxy · Road-graph node

- Detected via object detection + segmentation on citizen photos
- Severity from area, depth proxy, road occupancy
- Directly tied to a road_segment for GNN propagation

Garbage & Illegal Dumping

Vision · Recurrence tracking · Civic text

- Detected via a waste-pile / overflowing-bin detector class
- Severity from pile size, recurrence at the same spot, proximity to drains
- Strongest civic-text signal (complaints mention it most)

Unsafe / Unfenced Construction Sites

Vision · Multi-object reasoning

- Detects missing barricades, open pits, unsafe scaffolding, exposed rebar
- Severity from pit depth proxy, barricade presence, footfall proximity
- Highest weighting for pedestrian/vehicle exposure

Fallen Trees & Storm Debris

Vision · Weather-triggered

- Detects fallen/leaning trees and branch obstructions blocking a path
- Severity from % road-width blocked, tree size
- Forecast strongly correlated with the weather-risk feature (storms/high wind)

Exposed / Unfixed Electric Lines

Vision · Highest safety priority

- Detects hanging, sagging, or downed wires and damaged poles/transformers
- Treated as a critical-safety class — always floors the AURS score at a high minimum
- Auto-flagged for immediate human review, never left to routine queueing

Shared Intelligence Layer

One system underneath all five

- One fusion pipeline, one AURS score formula (class-aware weights)
- One road-graph GNN forecasting risk across all classes together
- One dispatch, one verification workflow, one command centre

Design implication

The vision model becomes a single multi-class detector (one backbone, five+ classes) rather than five separate models — this is both more realistic for a one-semester build and creates a genuinely interesting multi-task learning discussion for the viva: does sharing features across hazard classes improve or hurt per-class accuracy?

Problem, Motivation & Research Positioning

3.1 The Reactive Maintenance Problem

Municipal hazard management — for roads, waste, construction oversight, tree maintenance, and electrical infrastructure alike — is almost always reactive: a defect is noticed, reported through a disconnected channel, manually triaged, assigned informally, addressed at an arbitrary priority, and closed without independent verification. This creates four structural gaps that ARGUS targets directly, across all five hazard classes:

- **Duplication gap** — the same garbage pile or pothole is reported repeatedly as unrelated tickets.
- **Prioritisation gap** — severity and urgency are judged subjectively, with no consistent, auditable score — dangerous when an exposed wire and a minor pothole compete for the same crew.
- **Foresight gap** — action is triggered only after a hazard is reported, never before it forms from a visible pattern (a construction site left unfenced for weeks, a drain that floods every monsoon).
- **Verification gap** — closure is marked administratively, with no independent evidence the fix actually worked.

3.2 How Existing Approaches Fall Short

Category	Representative approach	Key limitation ARGUS addresses
Citizen complaint apps	Government grievance portals, generic "report-a-problem" apps	No AI triage, no deduplication, no spatial reasoning, no verification — a form, not an intelligence system
Academic hazard-detection papers	Single-modality CNN/YOLO detectors for one hazard type, on curated datasets	Stop at detection; no fusion across hazard classes, no forecasting, no loop back to dispatch/verification
Commercial municipal-asset platforms	Enterprise GIS + IoT sensor fleets	Require paid sensors, LiDAR vans, commercial GIS licences — infeasible for a student budget

3.3 Research Positioning & Novelty Statement

ARGUS's core academic claim is narrow and defensible: a hazard's effective risk is not independent of its neighbours on the road graph or of other hazard classes nearby. A pothole near an unfenced construction site, on a segment with recent garbage buildup blocking a drain, ahead of monsoon weather, is a compounding risk — not five separate tickets. This is a graph propagation problem, not a per-image classification problem.

Defensible novelty claim for the viva

"We do not claim five new hazard detectors. We claim that fusing multi-class hazard detections into a shared road-network graph and propagating risk with a GNN produces measurably better 7-day forecasts than treating each hazard, and each class, as independent — and we prove it with an ablation against three non-graph baselines on the same pilot dataset."

Objectives & Success Metrics

#	Objective	ARGUS capability	Success metric
O1	Multi-class multimodal fusion	Combine image, text, GPS, weather, GIS evidence into one hazard record, for any of the 5 classes	≥ 90% of test hazards carry all applicable evidence fields
O2	Hazard characterisation	Detect hazard type (5 classes) and generate a transparent, decomposable severity score	mAP ≥ 0.6 across classes; severity rank correlation ≥ 0.7 vs human raters
O3	Spatio-temporal prediction	Forecast 7-day / 30-day segment-level risk via GNN vs baselines, across all classes	GNN reduces MAE by a statistically significant margin over the best non-graph baseline
O4	Closed-loop verification	Classify repair/cleanup/fix state from before/after evidence, for any hazard class	Accuracy ≥ 0.8 on a held-out before/after set
O5	Passive civic mining	Extract hazard/location signals from public text when live feeds are unavailable	NER F1 ≥ 0.75 on an annotated civic-text sample
O6	Dynamic dispatch	Route a limited crew fleet against priority-weighted, mixed-class hazards	≥ 15% total-distance improvement over nearest-neighbour
O7	Immersive visualisation	3D digital twin with temporal playback across all hazard classes at once	Live demo runs end-to-end in under 5 minutes, no manual intervention
O8	Zero recurring cost	Build and demonstrate the entire pipeline with no paid API or managed cloud service	₹0 monthly spend, itemised in Section 17

Success condition

A new report of any hazard class travels through the complete pipeline — ingestion → geospatial normalisation → AI perception → fusion → GNN risk propagation → map → dispatch → verification → audit/history — with a measured metric captured at every stage.

System Architecture

ARGUS is a modular, local-first platform. Heavy inference runs on the developer machine (or a free-tier cloud GPU session for training only), so the live system never depends on a paid per-request provider. Services are split along domain boundaries so any model — including any single hazard-class detector head — can be replaced or retrained without touching the dashboard or schema.

Layered Reference Architecture

Presentation

Next.js + Tailwind + CesiumJS command centre; installable offline-capable PWA for citizen reporting and field crews.

API / Contract

FastAPI exposing a stable REST + WebSocket contract — the only layer the frontend ever talks to.

Intelligence

Five workers: vision, nlp, forecast (GNN + baselines), verification, routing.

Geospatial + Storage

PostgreSQL + PostGIS for all spatial reasoning; local disk or free-tier object storage for media.

Messaging

Redis for job queues and WebSocket pub/sub fan-out.

Deployment

Docker Compose locally; optional free-tier hosting for a public demo URL.

Event-Driven Flow

NEW_REPORT → schema + geofence validation → vision-worker inference (class + severity) → HAZARD_UPDATED → fusion + AURS → PRIORITY_UPDATED → GNN forecast refresh (batched) → optional dispatch → REPAIR_SUBMITTED → verification-worker → VERIFICATION_COMPLETED → case closed/reopened → audit_events append → nightly retraining snapshot.

Service Boundaries

Worker	Responsibility	Key libraries
vision-worker	Multi-class hazard detection (5 classes), severity feature extraction, depth proxy	PyTorch, torchvision, OpenCV, MiDaS
nlp-worker	Civic text NER, classification, urgency/sentiment cues	Hugging Face Transformers
forecast-worker	Feature engineering, baseline models, GNN training + inference	scikit-learn, XGBoost/LightGBM, PyTorch Geometric
verification-worker	Before/after alignment, class-aware fix classification	OpenCV, Siamese network in PyTorch
routing-worker	Priority-aware vehicle routing across mixed-class crews	OR-Tools, NetworkX

AI / ML Methodology — Per Hazard Class

PyTorch is the primary framework throughout. A single shared detector backbone is fine-tuned with five class heads, so the model learns general "something is structurally wrong here" features while still producing a class-specific severity path.

Hazard class	Detection approach	Severity features	Class-specific note
 Potholes / road damage	Object detection + segmentation mask	Affected area, shape irregularity, relative depth proxy, road occupancy	Depth is a relative cue only — never a certified physical measurement from one RGB photo
 Garbage / dumping	Object detection on waste piles / overflowing bins	Pile area, recurrence count at the same point, proximity to a drain or water body	Strongest signal actually comes from recurring civic-text complaints, not just images
 Construction site hazards	Multi-object detection: missing barricade, open pit, exposed rebar, unsafe scaffold	Pit depth proxy, barricade presence/absence, estimated pedestrian/vehicle footfall	Highest exposure weighting — treated as high-risk even at moderate severity
 Fallen trees / debris	Object detection for fallen/leaning trunks and branch obstructions	% of road width blocked, estimated tree size	Forecast is weather-triggered — spikes sharply ahead of storm/high-wind windows
 Exposed electric lines	Object detection for sagging/downed wires, damaged poles or transformers	Wire height above ground (proxy), proximity to pedestrian path	Critical-safety floor — AURS is clamped to a minimum "urgent" band regardless of other terms, and auto-routed to human review

6.1 Depth Estimation (shared)

MiDaS (small, CPU-friendly) or Depth Anything is used as a frozen, pretrained depth component supporting both pothole depth-proxy and construction-pit depth-proxy features — no depth-specific training data required.

6.2 NLP / Civic Signal Mining

Hugging Face Transformers (a distilled model fine-tuned on a small sample, or zero-shot where data is scarce) converts unstructured civic text into structured signals: hazard type (mapped to one of the five classes), location entity, urgency cue, sentiment/context, and extraction confidence. This channel is disproportionately useful for garbage/dumping complaints, which citizens report in text more often than they photograph.

6.3 Repair / Resolution Verification (shared, class-aware)

OpenCV performs perspective alignment of before/after images, combined with SSIM and a learned Siamese embedding for the final classification: Resolved / Partially Resolved / Not Resolved / Uncertain, with a confidence score. The "after" evidence differs meaningfully by class — a filled pothole, a cleared garbage point, a fenced pit, a removed tree, or a re-insulated wire — so the verification model is trained with class-conditioned examples of each.

6.4 Explainable AI Layer

- Severity and AURS decompose additively into named contributing factors, per class.
- The Stage-B tabular forecaster exposes SHAP feature importances per prediction.

- The GNN exposes attention weights showing which neighbouring segments — and which hazard classes on them — most influenced a forecast.

Routing & Dispatch Across Mixed Hazard Classes

Dispatch is formulated as a priority-aware capacitated vehicle-routing problem, now with a twist: crews are often class-specialised in real life (an electrical crew is not a garbage-collection crew). ARGUS models this directly.

- **Crew skill tags** — each crew record carries the hazard classes it can address (e.g. road-repair, sanitation, electrical, arboriculture, general-safety).
- **Constraint-aware assignment** — the router only assigns a hazard to a crew that can actually resolve it, then optimises route order within that constraint.
- **Cross-class escalation** — an exposed-wire hazard can trigger an immediate out-of-band alert to a human operator even if no electrical crew is currently free, rather than silently queueing behind lower-priority work.

Three tiers are implemented and compared: a nearest-neighbour baseline, an OR-Tools constraint-programming solver (free, Apache-2.0), and — as an optional stretch goal only, never a launch requirement — a reinforcement-learning dispatch policy.

5

HAZARD CLASSES

1

SHARED DETECTOR BACKBONE

₹0

MONTHLY RECURRING COST

20

WEEK BUILD PLAN

The Predictive Engine — Spatio-Temporal GNN

This is the project's headline research component, built in four escalating stages so a working forecaster exists at every checkpoint even if the most advanced stage runs out of semester time.

Stage A — Naive Baseline

Historical rate / moving average per road segment, per hazard class. Establishes the floor every later model must beat.

Stage B — Tabular ML

LightGBM or XGBoost on engineered features: recent hazard counts per class, weather exposure, traffic proxy, days since last resolution, seasonality. Strong, fast, and explainable via SHAP.

Stage C — Temporal Deep Learning

An LSTM or temporal convolutional network, included only if the pilot's time-series depth genuinely justifies it — evaluated fairly against Stage B, never assumed superior.

Stage D — Spatio-Temporal Graph Neural Network (headline)

Road segments are nodes; edges connect physically adjacent segments and segments sharing a drainage basin or historically correlated hazard patterns — **across all five hazard classes simultaneously**, so a cluster of new garbage points and a nearby unfenced construction pit can jointly raise the predicted risk of an untouched neighbouring segment. Built with PyTorch Geometric (free, open-source) using a Graph Convolutional or Graph Attention Network.

Why the GNN stage is the single highest-impact addition

It converts ARGUS from "a system that reacts to reports" into "a system that anticipates hazards — of any class — before citizens report them." That claim, backed by a clean four-stage ablation on one pilot dataset, is exactly what turns a good engineering demo into a defensible research contribution.

All four stages report the same metric set — MAE, RMSE, MAPE where relevant, and calibration/uncertainty — on an identical held-out time split, producing one clean comparison table rather than four disconnected experiments.

Fusion & the ARGUS Urban Risk Score (AURS)

8.1 Unified Hazard Record

Every hazard, regardless of class, carries: hazard ID, class, coordinates, source(s), timestamps, detection confidence, severity, recurrence count, road context, weather exposure, GNN forecast, estimated response effort, repair state, and evidence links. This is what a supervisor actually reads.

8.2 AURS Formula (Class-Aware)

$$\text{AURS} = w_1 \cdot \text{Severity} + w_2 \cdot \text{Exposure} + w_3 \cdot \text{Recurrence} + w_4 \cdot \text{WeatherRisk} + w_5 \cdot \text{GNN_Forecast} + w_6 \cdot \text{SourceConfidence} + \text{ClassFloor}(\text{class})$$

All terms normalise to 0–100, weights are documented in a versioned config file, and **ClassFloor** is a small addition specific to this version: it guarantees a hazard from a critical-safety class (exposed electric lines; deep, unfenced pits) can never be scored as low-priority purely because its other terms happen to be low.

Example Explainability Panel — Exposed Wire Hazard

Factor	Contribution
Critical-safety class floor (electric line)	+40
Severity (wire height / proximity to path)	+22
GNN 7-day forecast risk	+14
Multiple corroborating reports	+12
Weather risk (upcoming rain — electrical risk compounds)	+8
Source confidence	+4
TOTAL	= 100 / 100 (Urgent)

8.3 Duplicate & Corroboration Intelligence

Spatial clustering (PostGIS ST_ClusterDBSCAN), temporal proximity, and image-embedding similarity collapse corroborating reports of the same hazard — of any class — into one record with multiple evidence sources.

8.4 Citizen Trust Layer

A reporter reliability score (verified-report rate, false positives, duplicates, location accuracy) adjusts confidence gradually, and is designed so no single low-trust reporter can suppress or dominate a hazard's score.

Geospatial Intelligence Layer

PostGIS is the central spatial reasoning engine: spatial de-duplication, nearest-road-segment matching, hazard-to-segment linking, radius queries, DBSCAN hotspot clustering, temporal aggregation, and map-ready GeoJSON generation for the Cesium frontend — shared identically across all five hazard classes.

Multi-Resolution Evidence Strategy

Ground imagery

Fine-grained, per-hazard evidence — the primary detector input for all five classes.

Sentinel-2 (free tier)

Regional/environmental context only — vegetation and surface-water proxies, never a direct detector.

OpenStreetMap

Road-network topology and segment metadata, free with no key required at this scale.

OpenWeatherMap (free tier)

Weather features feeding AURS WeatherRisk and forecast-worker temporal features.

Open elevation / SRTM

Terrain for Cesium 3D and drainage-basin edges in the GNN's road graph.

Municipal ward boundaries

Open government shapefiles for reporting hazard density by administrative zone.

Hotspot Example — Compounding, Cross-Class

Cluster #17: an unfenced construction pit, 3 nearby garbage complaints, and 2 potholes within one 80m corridor, active for three weeks, with a GNN forecast flagging the adjacent untouched segment ahead of the monsoon window. ARGUS converts this from four isolated complaints across two departments into one recurring, cross-class infrastructure-risk hotspot — the kind of insight a manual complaint log structurally cannot produce.

Temporal Playback

A date slider on the command centre replays hazards of any class appearing, worsening, being resolved, and occasionally reappearing — making the predictive and closed-loop claims visually demonstrable in under a minute.

Dataset & Annotation Plan (Zero Cost)

A credible final-year project is judged as much on data discipline as on model choice. The dataset plan now spans five classes, but stays entirely free.

1. **Public seed datasets** — open road-damage datasets (RDD2022-style) for potholes; open waste/litter-detection datasets for garbage; general construction-site safety datasets where available. These bootstrap each class before campus-specific images exist.
2. **Campus-specific collection** — structured phone-camera walkthroughs of VIT Bhopal and Kothri on a fixed schedule (weekly, and immediately after rain or a storm), geotagged automatically. One walkthrough naturally captures multiple hazard classes in the same session — the zero-cost equivalent of a survey vehicle covering everything at once.
3. **Annotation** — CVAT (free, self-hostable via Docker) with a five-class label schema: bounding box/mask, hazard class, severity label, GPS, timestamp, road type, before/after pairing.
4. **Augmentation** — Alumentations (free) expands the small campus dataset with blur, low-light, rain, and compression variants — directly supporting the robustness evaluation, and especially relevant for the electric-line and fallen-tree classes, which are often photographed in poor weather.
5. **Civic text** — a small hand-annotated sample of public grievance-style text (garbage and electrical complaints dominate this channel), or a synthetic/replayed dataset where live feeds are unavailable.

Data quality principle

A smaller, carefully annotated pilot dataset across five honestly-labelled classes is worth far more — and is far more defensible in a viva — than a large, noisy, single-class scrape. State the exact number of annotated images per class, the annotation protocol, and inter-annotator agreement in the final report.

Per-Class Minimum Viable Dataset (suggested targets)

Class	Suggested min. annotated images	Primary source
Potholes / road damage	300–500	Public seed dataset + campus walkthroughs
Garbage / dumping	250–400	Campus walkthroughs + civic text corroboration
Construction site hazards	150–250	Campus/Kothri site visits (with permission)
Fallen trees / debris	100–200	Post-storm campus walkthroughs (event-driven)
Exposed electric lines	100–200	Campus walkthroughs + public safety-report datasets

Security, Privacy & Ethical Design

A system ingesting citizen-submitted geotagged photos of public infrastructure carries real privacy obligations. Addressing them explicitly is exactly the kind of maturity that distinguishes a strong final-year project from a purely technical demo — and it matters more, not less, now that construction sites and electrical infrastructure are in scope.

Data Minimisation & Anonymisation

- Faces and licence plates incidentally captured are automatically blurred locally before storage — never sent anywhere unblurred.
- Precise reporter identity is decoupled from public-facing hazard records; only an internal reliability score is retained.
- Raw GPS is snapped to the nearest road segment for public display; exact coordinates live only in the internal audit trail.

Access Control

- Role-based access: citizen, class-specialised field crew, municipal operator, read-only analytics.
- All state-changing API calls are authenticated and written to the immutable `audit_events` table.

Model & System Robustness

- Every AI output carries a confidence score; the UI visually distinguishes an AI prediction from a human-confirmed observation.
- Human override is always available — critically, for the electric-line and construction-site classes, ARGUS recommends urgency but never claims to make anything electrically or structurally safe by itself.

Responsible Scope

ARGUS never claims certified physical measurements from a single RGB image, never fully automates crew dispatch without a human-confirmable override, and treats every predictive output as a forecast with reported uncertainty — not a guarantee. For the electric-line class specifically, ARGUS is explicitly a **detection and prioritisation aid for human electrical crews**, not a safety-certification system.

Zero-Cost Technology Stack

Every layer has a free or local substitute with no recurring cost, and the pipeline keeps functioning if any single free tier is temporarily unavailable.

Layer	Free / local choice	Limit to know	Fallback
Deep learning	PyTorch, local or free Colab/Kaggle GPU	Colab sessions time out ~12h	Checkpoint + resume
Vision utility	OpenCV, Albumentations	None — fully local	—
NLP	Hugging Face Transformers	Fine-tuning needs a GPU session	CPU inference for distilled models
Graph learning	PyTorch Geometric	None — fully local	—
Prediction baselines	scikit-learn, XGBoost/LightGBM	None — fully local	—
Database	PostgreSQL+PostGIS (self-host); Neon/Supabase free tier for demo	~0.5–1GB free storage	Prune old media; keep metadata + thumbnails
Object storage	Local disk; Cloudflare R2 free allowance	10GB / 1M reads / mo	Stay well under pilot scale
Cache / jobs	Redis via Docker	None — fully local	—
Maps	OpenStreetMap	Fair-use tile limits	Self-host tiles / cache
Satellite / GIS	Sentinel-2 / Copernicus free tier	Monthly request quota	Cache regional context once
Weather	OpenWeatherMap free tier	60 calls/min, 1,000/day	Batch + cache per segment/day
3D visualisation	CesiumJS + Cesium ion free tier	Monthly streaming quota	Offline/local terrain tiles for demo
Routing	Google OR-Tools	None — Apache-2.0	—
Hosting (optional)	Render/Railway free tier	Sleeps after inactivity	Fine for a scheduled demo
CI/CD	GitHub Actions free minutes	Unlimited on public repos	Keep repo public
Packaging	Docker + Compose	None — fully local	—

Cost-control rule

APIs are used only where they add clear value and stay within a documented free tier. Core perception, scoring, forecasting, and routing never depend on a paid per-request provider.

Data Model & API Blueprint

Core Tables

Table	Purpose
hazards	Canonical hazard entity + geometry + class — the fused, de-duplicated record
observations	Individual raw citizen/social observations, pre-fusion
media	Image evidence, linked to observations and repairs
hazard_scores	Severity / confidence / AURS outputs, versioned per recalculation
weather_observations	Historical/forecast weather context per segment
road_segments	OSM-derived road graph nodes/edges, used directly by the GNN
predictions	Time-indexed forecasts from every model stage (A–D)
crews	Crew availability, location, capacity, and class skill tags
dispatch_runs	Routing optimisation outputs and their scoring metrics
repairs	Fix/repair/cleanup action records, linked to before/after media
verifications	Before/after AI verification results and confidence
users / reputation_events	Accounts and the citizen trust layer
audit_events	Immutable lifecycle trail for every state-changing action

Representative API Surface

- `POST /reports` — submit a new observation of any hazard class
- `GET /hazards`, `GET /hazards/{id}`, `GET /hazards/nearby?lat=&lon=&radius=&class=`
- `GET /hotspots` — DBSCAN-clustered active hotspots, optionally cross-class
- `GET /risk/forecast?segment_id=&horizon=` — Stage A–D forecast comparison for a segment
- `GET /crews?skill=`, `POST /dispatch/optimize`
- `POST /repairs/{id}/verify`
- `GET /analytics` — executive dashboard aggregate metrics, per class and overall

WebSockets publish hazard-created, priority-updated, crew-moved, repair-submitted, and verification-completed events so the 3D command centre updates live without a manual refresh.

Evaluation & Research Plan

Subsystem	Baseline	Primary metrics	Ablation / comparison
Hazard detection (5 classes)	Pretrained off-the-shelf, unmodified	Precision, Recall, F1, mAP per class	Fine-tuned vs. pretrained-only; shared backbone vs. per-class model
Severity scoring	Rule-based score from area/shape only	MAE, RMSE, rank correlation vs. human raters	With vs. without depth-proxy feature
NER / location extraction	Keyword-matching baseline	Precision, Recall, F1, location accuracy	Zero-shot vs. fine-tuned
Forecasting (Stage A–D)	Moving average / historical rate	MAE, RMSE, MAPE, calibration	Naive vs. LightGBM vs. LSTM vs. GNN — the headline comparison
Repair/fix verification	SSIM / classical difference only	Accuracy, Precision, Recall, F1	Classical-only vs. classical + Siamese embedding
Routing	Nearest-neighbour	Total distance, response time, priority-weighted coverage	Nearest-neighbour vs. OR-Tools vs. (optional) RL
System performance	Synchronous single-request baseline	API latency, inference time, throughput	Sequential vs. queued worker pipeline
Robustness	Clean, well-lit images only	Performance under blur, low light, rain, compression	Clean vs. augmented test set

The single most important table in the final report is the forecasting comparison (Stage A–D) on an identical held-out time split, reported with confidence intervals or a simple significance test — the evidence behind the headline novelty claim.

20-Week Implementation Roadmap

Weeks	Phase	Build	Exit criterion
1–2	Foundation	Repo, Docker Compose, FastAPI skeleton, PostGIS schema (5-class ready), Next.js + Cesium shell	Empty map renders; API health-check responds
3–4	Ingestion & PWA	Citizen PWA (offline-queue), NEW_REPORT endpoint (class selector), media storage	A submitted report of any class appears on the map
5–8	Perception	Shared-backbone detector fine-tuning on all 5 classes, severity pipelines per class	Each class shows AI-detected type, severity, confidence
9–10	Fusion & dedup	Spatial clustering, corroboration logic, AURS v1 with class floors	Duplicate/cross-class reports collapse into ranked hazards
11–13	Prediction I–II	Weather/temporal features, Stage A + Stage B forecasting	7-day forecast with reported MAE/RMSE per segment
14–16	Prediction III — GNN	Road-graph construction, PyTorch Geometric GCN/GAT, Stage C/D comparison	GNN forecast beats baselines with a reported margin
17	Prescriptive layer	OR-Tools routing with crew skill tags, priority-weighted coverage	Mixed-skill crews routed against 20+ mixed-class hazards
18	Closed loop	Repair workflow, class-aware verification worker, audit trail	A hazard of any class can be verified or reopened end-to-end
19	Command centre polish + evaluation	Temporal slider, explainability panels, what-if simulator, full metrics table	Every subsystem has a reported baseline-vs-final metric
20	Documentation & demo rehearsal	Final report, viva slides, rehearsed 5-minute demo across all 5 classes	Demo runs live, twice, with no manual intervention

Scope discipline

Kubernetes, full city-scale deployment, exact cost estimation, fully autonomous field robotics, and 24/7 uptime remain future-scale concepts, not requirements. A dockerized modular system, reproducible experiments, and a convincing five-minute pilot demo are the priority.

Deliverables & Live Demo Script

Citizen View

Capture/upload a geotagged photo of any hazard class; see local AI feedback; submit; track status to verified resolution.

Authority Command Centre

3D Cesium map of all live hazards across classes, risk heatmap layered with GNN forecast zones, crew locations/routes, per-hazard explainability panel.

Field Crew View

Class-filtered assigned route, priority task list, navigation, fix-status update, after-image submission — usable offline.

Verification View

Before/after comparison with resolution state, confidence, and explainable evidence, for any class.

Executive Analytics

Infrastructure Health Index, active hazards by class, resolution rate, response time, route efficiency, hotspots.

What-If Simulator

Adjust crew count/skill mix or weather assumptions; see the effect on response time, coverage, and forecast risk.

Suggested 5-Minute Demo Script

1. Open the command centre — show all five hazard classes live on the 3D map at once.
2. Submit one new report on the citizen PWA (e.g. a pothole) — watch it appear on the map within seconds.
3. Open its explainability panel — walk through the AURS breakdown.
4. Open the GNN forecast view — show the Stage A–D comparison table and point at a forecasted hotspot the model flagged before it was reported.
5. Trigger dispatch — show a class-skilled crew routed to it.
6. Submit an after-photo — show the verification worker classify it Resolved, and the case close in the audit trail.

Risk Register & Zero-Cost Proof

Risk	Why it matters	Mitigation
Insufficient labelled images per class	Rarer classes (fallen trees, exposed wires) may overfit	Prioritise data collection for rare classes; augment aggressively; report per-class metrics honestly
Monocular depth scale ambiguity	Exact depth can't be guaranteed from one RGB image	Use relative depth only; label as an estimate everywhere
Sparse time-series history	LSTM/GNN may be under-justified early on	Use strong classical baselines regardless; report GNN result with honest data-volume caveats
Class imbalance in the shared detector	Common classes (potholes) could dominate training	Class-balanced sampling / loss weighting; monitor per-class recall
Live social-media access	API access, policy, or availability may change	Treat passive mining as modular; support replayed/synthetic data
Free-tier quota exhaustion	A demo could hit a rate limit at the worst moment	Cache aggressively, batch external calls, rehearse against cached data
False confidence in AI outputs	Incorrect recommendations could be trusted blindly — especially risky for electrical hazards	Always show confidence/evidence; human override at every decision point

Itemised ₹0 Proof

Component	Monthly cost	Justification
Compute (dev + optional GPU boost)	₹0	Own machine; free Colab/Kaggle GPU sessions
Database, object storage	₹0	Self-hosted, or Neon/Supabase/R2 free tiers
Maps, satellite, weather	₹0	OpenStreetMap, Sentinel/Copernicus, OpenWeatherMap free tiers
3D visualisation, routing	₹0	Cesium ion free tier / local tiles; Google OR-Tools
Hosting, CI/CD	₹0	Render/Railway free tier; GitHub Actions free minutes
TOTAL RECURRING COST	₹0	Every dependency has a documented free tier or is self-hosted

Final Vision & Recommended Title

ARGUS is not a pothole app, a complaint tracker, or a single detection model wrapped in a dashboard. It is one closed-loop infrastructure intelligence system that watches a pilot area for five real, co-occurring hazard classes, converts every observation into a spatially grounded, explainable record, propagates risk across the road network with a graph neural network to anticipate emerging hotspots before they're widely reported, dispatches the right crew for the right hazard, verifies the fix actually worked, and feeds every outcome back into the model that makes the next forecast better.

The final demonstration makes this loop obvious in under five minutes, across all five hazard classes at once: **report** → **detect** → **score** → **forecast** → **dispatch** → **fix** → **verify** → **learn** — and every word in that sentence corresponds to a working, evaluated piece of the system.

Recommended Project Title

"ARGUS: A Multimodal Geospatial Intelligence and Graph-Neural Predictive Decision-Support System for Closed-Loop Urban Infrastructure & Public-Safety Management." The wording reflects both the operational decision-support role and the graph-learning research contribution, while leaving room to discuss RL routing and edge inference as acknowledged future work.

Closing note for the supervisor meeting

Lead with the loop, not the tech stack: show one hazard — pick whichever class is visually strongest in your demo data — travelling from a citizen's photo to a verified fix live, then open the explainability panel and the GNN forecast comparison table. The five-class scope and the free-tier discipline prove it is buildable in one semester; the GNN ablation table proves it is more than a semester project.

END OF PROJECT SPECIFICATION — v3.0

Prepared as the working technical blueprint for ARGUS.